

Хранение данных, Content Provider

Осень 2025

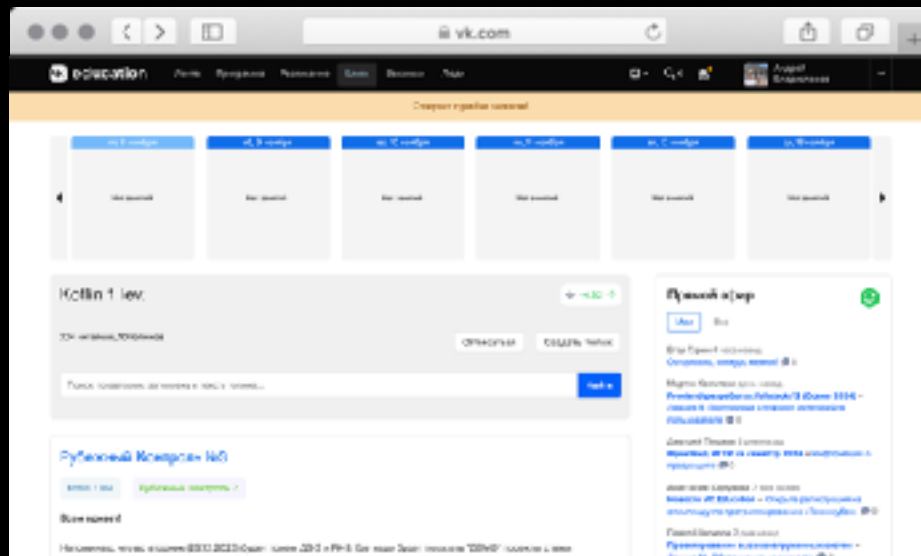




Андрей Владиславов

- Старший Android разработчик @ VK
- Занимаюсь SDK VK Клипов для разных продуктов компании
- Закончил Бауманку, кафедру ИУ9 в 2023

Не забудьте отметиться



План лекции

- Зачем?
- savedInstanceState
- SharedPreferences
- DataStore
- SQLite
 - + ORM
- File Storage
- AccountManager
- ContentProvider

Зачем?

Кэширование

Сохранение данных во избежание повторных запросов

Хранение данных

Сохранение пользовательских данных для переноса между сессиями

Хранение конфигураций

Сохранение пользовательских настроек (например, тема)

Пользовательский контент

Черновики, рисунки

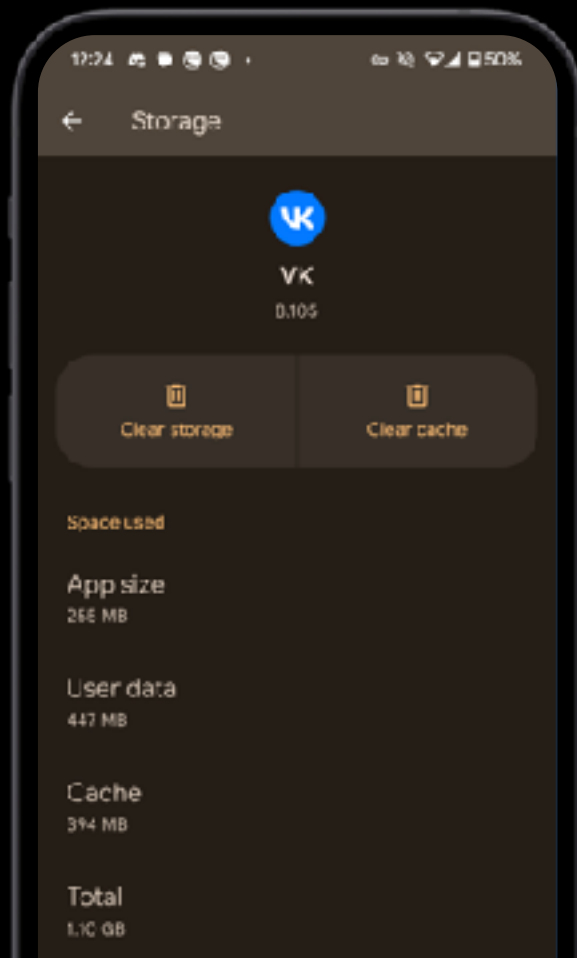
Авторизационные данные

Токены

Что-то ещё?

Но есть нюансы

- Пользователь может очистить кэш
- Пользователь может очистить данные приложения
- Безопасность данных
- Рост количества данных
- Обеспечения доступа к данным



savedInstanceState

На случай, если забыли

```
class MyFragment : Fragment() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
    }  
  
    override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
        super.onCreateView(view, savedInstanceState)  
    }  
  
    override fun onSaveInstanceState(outState: Bundle) {  
        super.onSaveInstanceState(outState)  
    }  
}
```

```
class MyActivity : Activity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
    }  
  
    override fun onSaveInstanceState(outState: Bundle) {  
        super.onSaveInstanceState(outState)  
    }  
}
```


Saved Instance State

Эти данные обычно используется для хранения состояния UI вашего экрана.

Но в этот “стэйт”, так же можно вкладывать и дополнительные параметры экрана (например, идентификаторы, или данные с которыми был открыт экран, или данные, которые экран приобрел для работы).

Несмотря на то, что в документации данные хранятся “в памяти”, при убивании вашего приложения в фоне, эти данные сохраняются в персистетнтном хранилище, в результате чего, хранящиеся внутри данные должны быть сериализуемы.

Доступ к нему вам предоставляет Activity и Fragment.

	ViewModel	Saved instance state
Storage location	in memory	in memory
Survives configuration change	Yes	Yes
Survives system-initiated process death	No	Yes
Survives user complete activity dismissal/onFinish()	No	No
Data limitations	complex objects are fine, but space is limited by available memory	only for primitive types and simple, small objects such as String
Read/write time	quick (memory access only)	slow (requires serialization/deserialization)

SharedPreferences

SharedPreferences

- Key-value хранилище данных
- Ключ – строка
- Может хранить
 - Прimitives типы данных (Boolean, Int, Long, Float)
 - Строки
 - Множества строк (Set<String>)
- Имеет два метода для сохранения
 - commit – синхронная запись. *!! Блокирующая операция, не использовать на главном потоке !!*
 - apply – асинхронная запись. Есть риски при смерти процесса
- По умолчанию создаётся с Context.MODE_PRIVATE
- Можно найти файл по пути *data/data/package.name/shared_prefs*. Пишется в xml

```
val prefs = context.getSharedPreferences(name, Context.MODE_PRIVATE)

prefs.getString(stringKey, /*default = */"NONE")
prefs.getInt(intKey, /*default = */-1)

prefs.edit().apply { // SharedPreferences.Editor
    putInt(intKey, 10)
    putString(stringKey, "MyValue")
    apply()
}
```

А что с
безопасностью?

Можно добавить шифрование

- Данные для шифрования храним отдельно
- Увеличивается время выполнения операций
- Можно пользоваться EncryptedSharedPreferences
- Возникает проблема хранения ключа шифрования

```
val masterKey = new MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build()

val sharedPreferences = EncryptedSharedPreferences.create(
    context,
    "secret_shared_prefs",
    masterKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
)

// use the shared preferences and editor as you normally would
val editor = sharedPreferences.edit();
```

Android Keystore System

Для безопасного хранения ключей на устройстве. Гарантирует сохранность материала, даже если процессы приложения будут скомпрометированы.

Можно управлять политикой использования ключей.

... но это лучше уже изучать в рамках безопасности приложения.

Для упрощения работы с таким хранилищем - лучше обратить внимание на фреймворк **google tink**.

```
val kpg: KeyPairGenerator = KeyPairGenerator.getInstance(  
    KeyProperties.KEY_ALGORITHM_EC,  
    "AndroidKeyStore"  
)  
val parameterSpec: KeyGenParameterSpec = KeyGenParameterSpec.  
    .Builder(alias, KeyProperties.PURPOSE_SIGN or KeyProperties.PURPOSE_VERIFY).run {  
        setDigests(KeyProperties.DIGEST_SHA256, KeyProperties.DIGEST_SHA512)  
        build()  
    }  
  
kpg.initialize(parameterSpec)  
val kp = kpg.generateKeyPair()  
  
/*  
 * Verify a signature previously made by a private key in the  
 * KeyStore. This uses the X.509 certificate attached to the  
 * private key in the KeyStore to validate a previously  
 * generated signature.  
 */  
val ks = KeyStore.getInstance("AndroidKeyStore").apply {  
    load(null)  
}  
val entry = ks.getEntry(alias, null) as? KeyStore.PrivateKeyEntry  
if (entry == null) {  
    Log.w(TAG, "Not an instance of a PrivateKeyEntry")  
    return false  
}  
  
val valid: Boolean = Signature.getInstance("SHA256withECDSA").run {  
    initVerify(entry.certificate)  
    update(data)  
    verify(signature)  
}
```

Чуть больше про шифрование

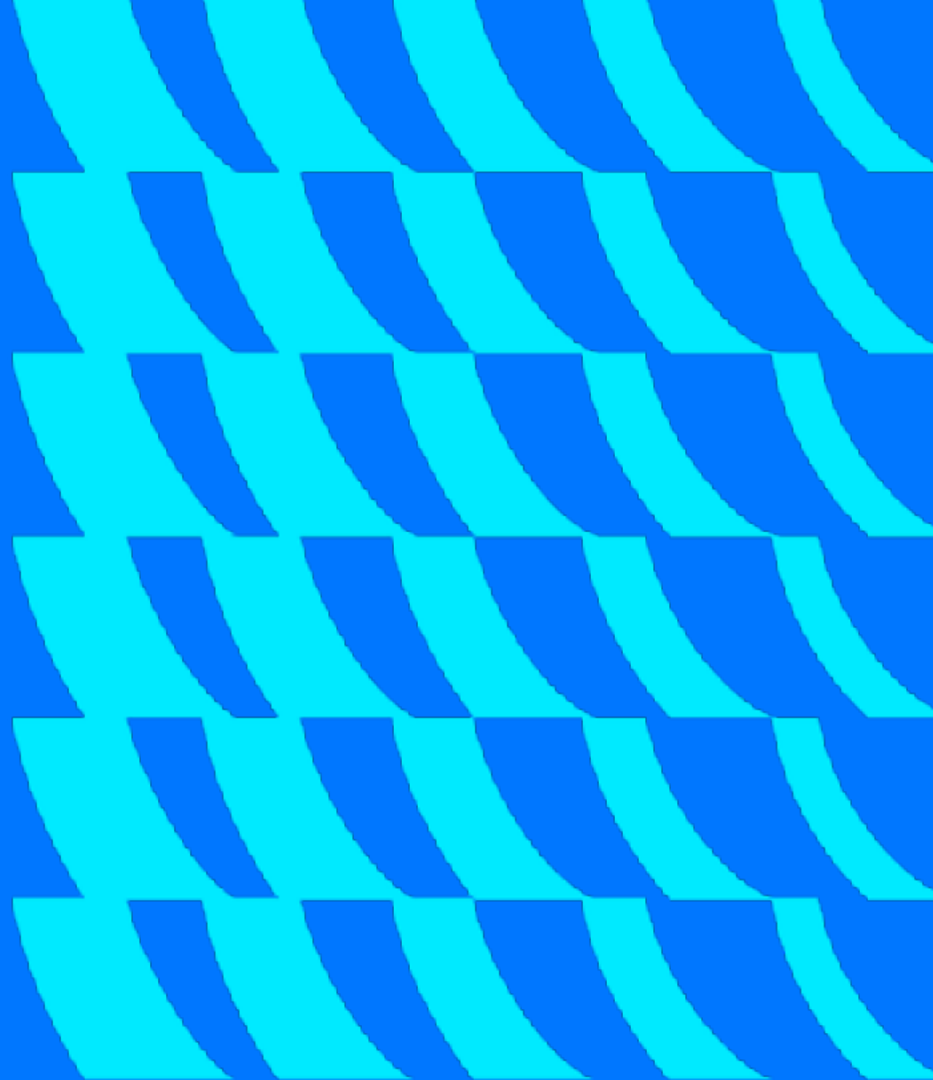


Шифруем файлы в Android
([Доклад на Mobius](#))



Обзор актуальных
инструментов шифрования
([habr](#))

DataStore



DataStore

- Preferences DataStore и Proto DataStore
- Типизированный ключ
- Поддерживает работу с Kotlin Coroutines по умолчанию и gxJava через интероп зависимость
- Пишутся в своём формате в Можно найти файл по пути *data/data/package.name/files/datastore/*

Preferences DataStore

```

// top-level declaration
val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "vk_edu")

val studyYearKey = IntPreferencesKey("study_year")

viewModelScope.launch {
    datastore.updateData { prefs ->
        val mutablePrefs = prefs.toMutablePreferences()
        mutablePrefs[studyYearKey] = 2024
        mutablePrefs // returning
    }
}

viewModelScope.launch {
    datastore.data.collect { prefs ->
        val studyYearKey = prefs[studyYearKey]
        updateState(studyYearKey)
    }
}
```

Proto DataStore

- Описать объект в *app/src/main/proto/* в формате ProtoBuf

Класс сгенерируется после компиляции

- Написать сериализатор и extension к Context

```
syntax = "proto3";

option java_package = "com.example.application";
option java_multiple_files = true;

message Settings {
    int32 example_counter = 1;
}
```

```
object SettingsSerializer : Serializer<Settings> {
    override val defaultValue: Settings = Settings.getDefaultInstance()

    override suspend fun readFrom(input: InputStream): Settings {
        try {
            return Settings.parseFrom(input)
        } catch (exception: InvalidProtocolBufferException) {
            throw CorruptionException("Cannot read proto.", exception)
        }
    }

    override suspend fun writeTo(
        t: Settings,
        output: OutputStream
    ) = t.writeTo(output)
}

val Context.settingsDataStore: DataStore<Settings> by datastore(
    fileName = "settings.pb",
    serializer = SettingsSerializer
)
example_counter = 1;
}
```

Proto DataStore

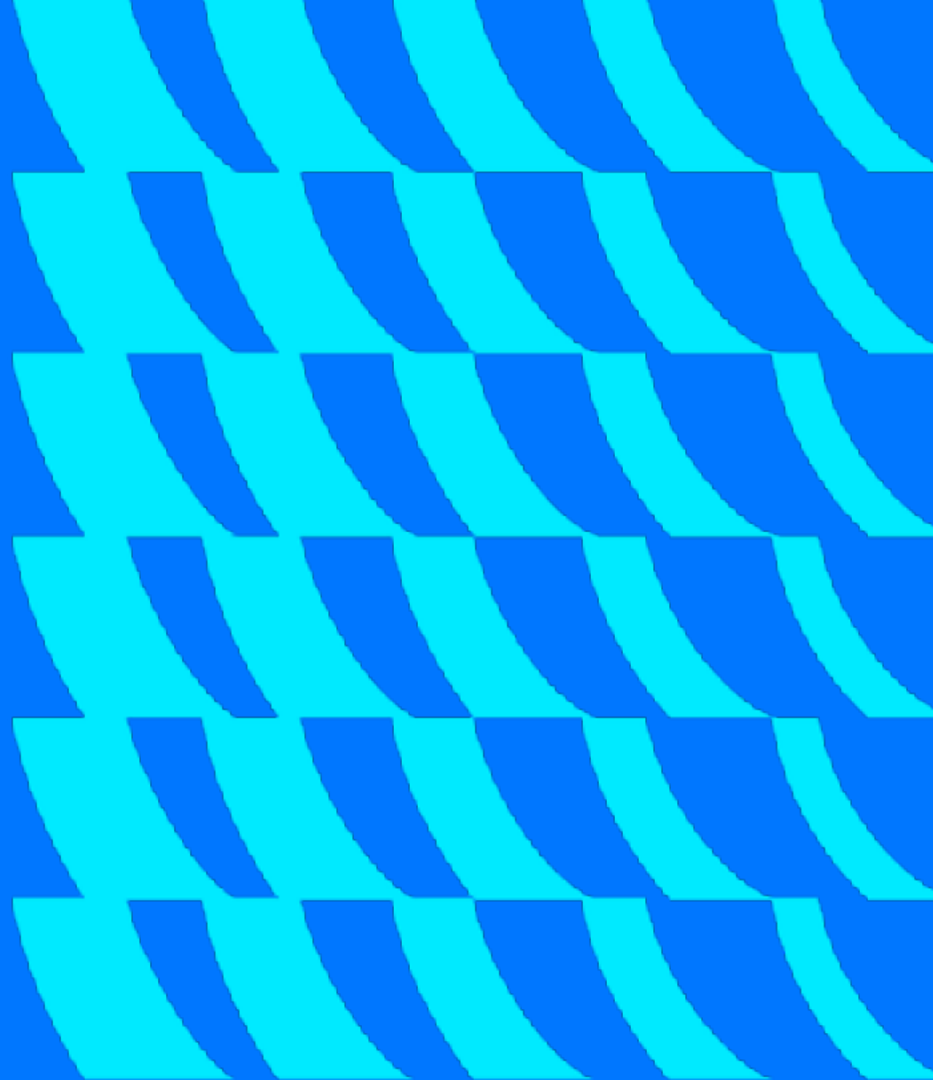


```
val exampleCounterFlow: Flow<Int> = context.settingsDataStore.data
    .map { settings ->
        // The exampleCounter property is generated from the proto schema.
        settings.exampleCounter
    }

suspend fun incrementCounter() {
    context.settingsDataStore.updateData { currentSettings ->
        currentSettings.toBuilder()
            .setExampleCounter(currentSettings.exampleCounter + 1)
            .build()
    }
}
```

Feature	SharedPreferences	PreferencesDataStore	ProtoDataStore
Async API	✓ (only for reading changed values, via listener)	✓ (via Flow and RxJava 2 & 3 Flowable)	✓ (via Flow and RxJava 2 & 3 Flowable)
Synchronous API	✓ (but not safe to call on UI thread)	✗	✗
Safe to call on UI thread	✗(1)	✓ (work is moved to Dispatchers.IO under the hood)	✓ (work is moved to Dispatchers.IO under the hood)
Can signal errors	✗	✓	✓
Safe from runtime exceptions	✗(2)	✓	✓
Has a transactional API with strong consistency guarantees	✗	✓	✓
Handles data migration	✗	✓	✓
Type safety	✗	✗	✓ with Protocol Buffers

SQLite



Hello SQLite

В Android из коробки есть одна база данных - это SQLite.

Она имеет ряд ограничений, и ее версия зависит в основном от версии Android. В результате чего, официальная документация по возможностям DB может применяться не ко всем Android.

Но если не требуется сложных операций, то может быть достаточным использование базовых функций.

Традиционно считается, что DB быстрее обрабатывает при записи и поиске данных (время выполнения операции слабо зависит от количества данных за счет индексирования).

Для работы с SQLite традиционно используется SQLiteOpenHelper, а данные представляются в виде Cursor.

От разработчика будет требоваться знание SQL, и внутренних объектов Android, которые помогают работать с SQLite.

```
class PlateSqlHelper(context: Context) : SQLiteOpenHelper(context, "SqlAccesser", null, 1) {
    override fun onCreate(db: SQLiteDatabase?) {
        if (db == null) {
            return
        }

        db.execSQL("CREATE TABLE PLATE")
    }

    override fun onUpgrade(db: SQLiteDatabase?, oldVersion: Int, newVersion: Int) {
        if (db == null) {
            return
        }

        db.execSQL("DROP TABLE PLATE")
        onCreate(db)
    }

    fun plates(): List<Plate> {
        return readableDatabase.use { db ->
            db.rawQuery("SELECT * FROM PLATE", null).use { cursor -> cursor.convert() }
        }
    }
}
```

Делаем проще

ORM (object-relational mapping)

Объектно-реляционное сопоставление :)

Простыми словами – за счет высокоуровневого описания объектов, таблиц и связей, фреймворк за вас будет сам понимать, какие требуется делать запросы для получения/сохранения данных.

Примеры фреймворков: ORMLite, GreenDAO, ObjectBox



```
// Пример использования ObjectBox
@Entity
data class User{
    @Id
    var id: Long = 0,
    var name: String? = null
}
...

try {
    box.put(User("Sam Flynn"))
} catch {e: UniqueViolationException} {
    // A User with that name already exists.
}
...

val query = box.query().run {
    equal(property, value)
    order(property)
    build()
}
```

Room

Фреймворк от Google, который пишет запросы для SQLite за вас.

В 100% случаях за вас скрипт фреймворк не напишет, поэтому знания SQL желательны. Но для простых структур - достаточно знание работы самого фреймворка.

Работа фреймворка основывается на большом количестве аннотаций, при помощи которых описываются сущности и интерфейсы.

```
// Пример описания структуры Base SQL
@Entity
table = "statistic",
indices = [Index("id")],
foreignKeys = [
    ForeignKey(
        entity = ResultsDbEntity::class,
        parentColumns = ["id"],
        childColumns = ["result_id"]
    ),
    ForeignKey(
        entity = DifficultyLevelsDbEntity::class,
        parentColumns = ["id"],
        childColumns = ["difficulty_id"]
    )
]

data class StatisticDbEntity(
    @PrimaryKey(autoGenerate = true) val id: Long,
    @ColumnInfo(name = "result_id") val resultId: Long,
    @ColumnInfo(name = "difficulty_id") val difficultyId: Long,
    val mistakes: Long,
    val points: Long
)

@Entity(tableName = "results")
data class ResultsDbEntity(
    @PrimaryKey val id: Long,
    @ColumnInfo(name = "result_name") val resultName: String
)

@Dao
interface StatisticDao {
    @Insert(entity = StatisticDbEntity::class)
    fun insertStatisticData(statistic: StatisticDbEntity)

    @Query("SELECT statistic.id, result_name, difficulty_name, mistakes, points FROM statistic * " +
        "INNER JOIN results ON statistic.result_id = results.id" +
        "INNER JOIN difficulty_levels ON statistic.difficulty_id = difficulty_levels.id")
    fun getAllStatisticData(): List<StatisticInfoTuple>
}
```

SQLDelight

Фреймворк для работы с базами данных, разработанная командой Square, которая позволяет генерировать типобезопасный Kotlin-код на основе SQL-запросов.

В отличие от Room, структура таблиц описывается в SQL-like формате и по описанию уже генерируются Kotlin классы

Инструмент получил популярность в том числе из-за поддержки KMP, но сейчас Room уже догнал SQLDelight в этом плане

SQLDelight

(1) MyDatabase.sq

```
CREATE TABLE User (  
  id INTEGER PRIMARY KEY,  
  name TEXT NOT NULL  
);  
  
-- Получить всех пользователей  
selectAll:  
SELECT * FROM User;  
  
-- Вставить нового пользователя  
insertUser:  
INSERT INTO User (id, name) VALUES (?, ?);
```

(2) build.gradle.kts

```
sqlDelight {  
  database("MyDatabase") {  
    packageName = "com.example.database"  
  }  
}
```

(3) Application code

```
import com.example.database.MyDatabase  
import com.squareup.sqlDelight.android.AndroidSqliteDriver  
  
val driver = AndroidSqliteDriver(MyDatabase.Schema, context, "mydatabase.cb")  
val database = MyDatabase(driver)  
  
// Вставить данные  
database.userQueries.insertUser(1, "John Doe")  
  
// Получить всех пользователей  
val users = database.userQueries.selectAll().executeAsList()  
users.forEach {  
  println("User: ${it.name}")  
}
```

Hello NoSql

Не реляционная структура:)

Для не супер сложных зависимостей - одно из лучших решений. Гибкая структура, минимальные описания структур. Для Android такие фреймворки вообще выглядят как key-value хранилища.

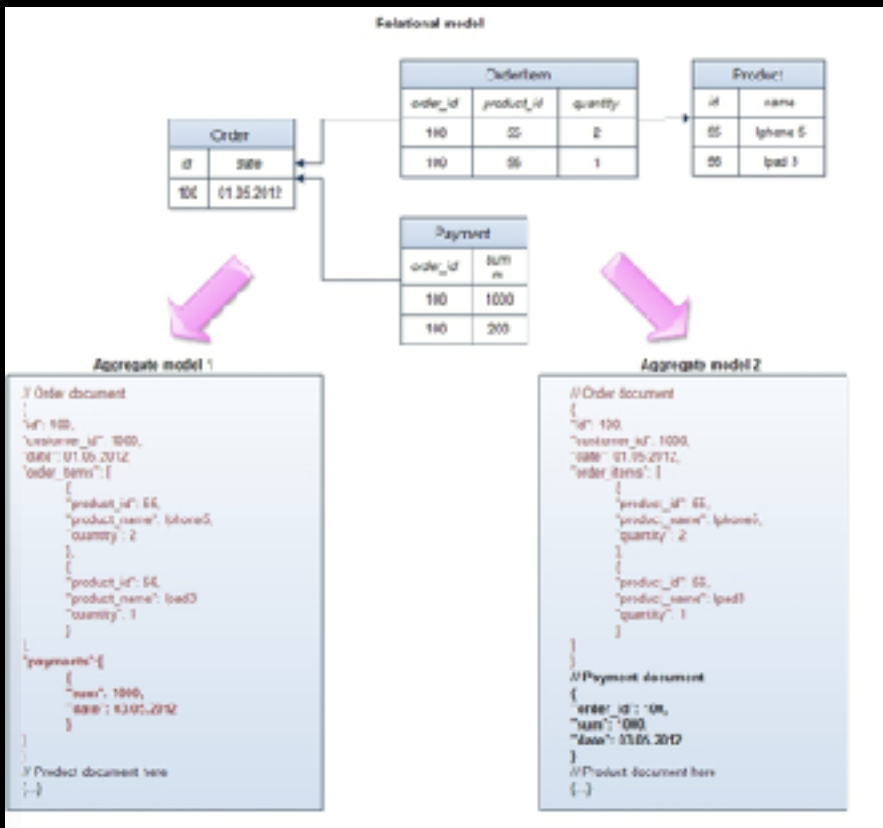
Вне Android тоже достаточно популярный подход. Как пример - MongoDB.

Примеры фреймворков: realm, OrmLite, PaperDB, Hawk
Firebase Database также является NoSQL базой

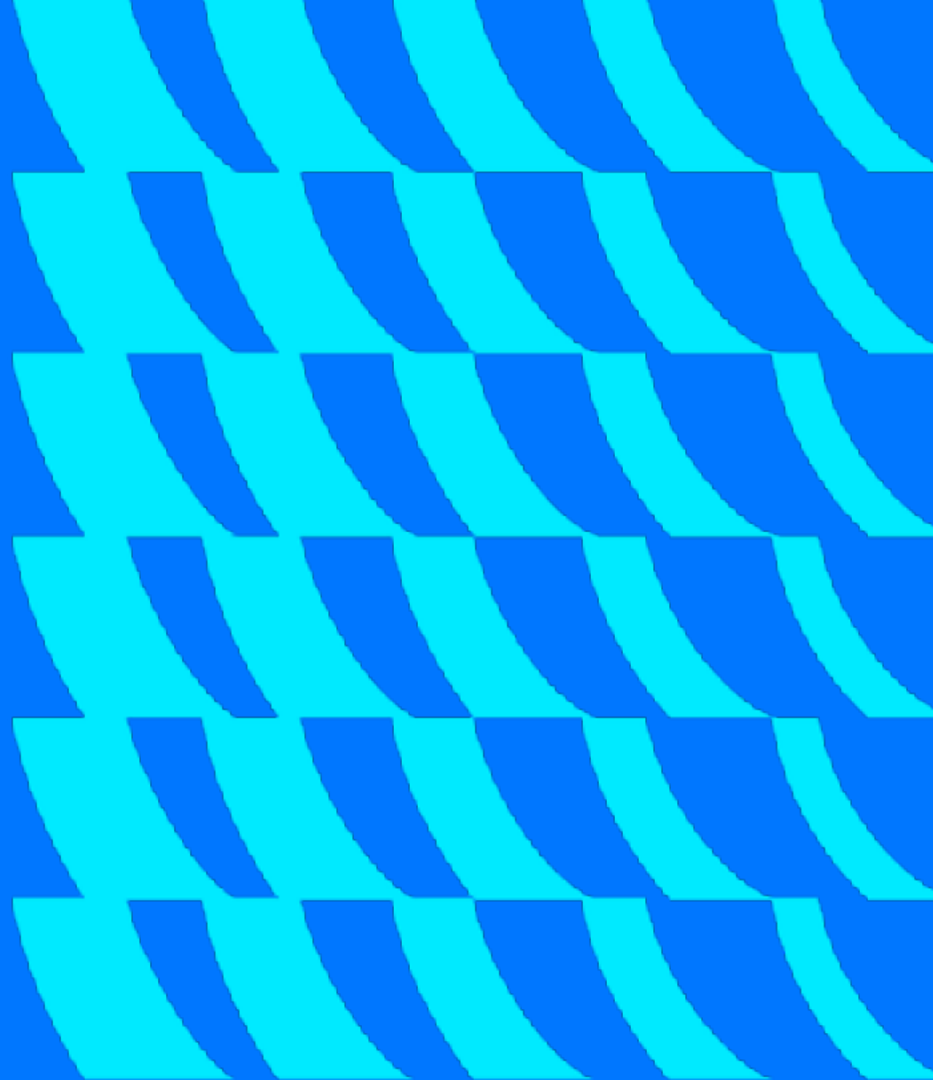


```
// Пример записи на PaperDB
Paper.book("for-user-1").write("contacts", contacts)
Paper.book("for-user-2").write("contacts", contacts)

val contacts = Paper.book().read("contacts")
```



FileStorage



External Storage

Для начала...

Необходимо проверить состояние внешнего хранилища:

- `Environment.getExternalStorageState()`

Возможные состояния:

- `Environment.MEDIA_MOUNTED`
- `Environment.MEDIA_MOUNTED_READ_ONLY`

Внешнее хранилище

Начиная с Android API 30 надо уже проверять разрешение, для получения свободного доступа:

- `Environment.isExternalStorageManager()`

Или использовать общие хранилища:

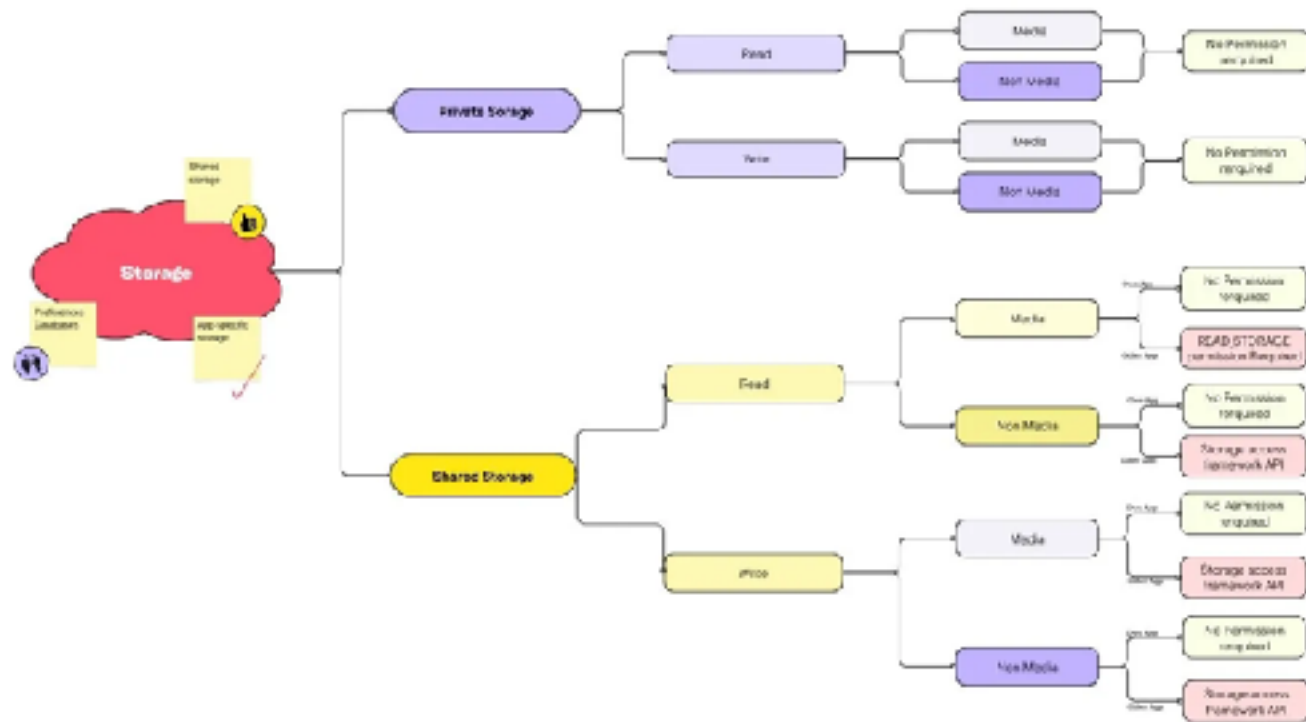
```
<uses-permission
    android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="29"
/>

<uses-permission
    android:name="android.permission.MANAGE_EXTERNAL_STORAGE"
    android:minSdkVersion="30"
/>
```

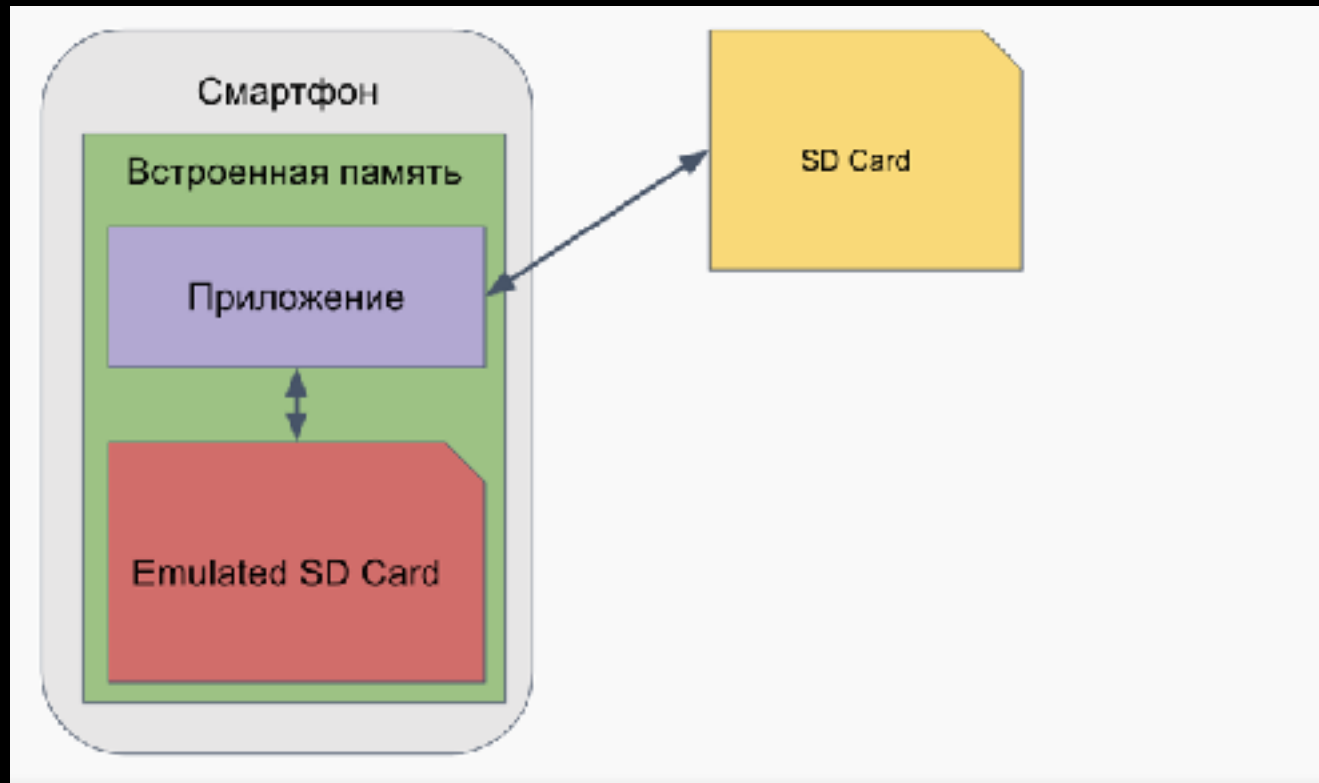
```
fun isExternalStorageWritable(): Boolean {
    val state = Environment.getExternalStorageState()
    return Environment.MEDIA_MOUNTED == state
}

fun isExternalStorageReadable(): Boolean {
    val state = Environment.getExternalStorageState()
    return Environment.MEDIA_MOUNTED == state
        || Environment.MEDIA_MOUNTED_READ_ONLY == state
}
```

Далее уже смотреть на Storage Access Framework



Внешнее хранилище, иногда не внешнее..



Internal Storage

Простыми словами - доступ к файлам имеет только ваше приложение. В общем случае - никто в эти файлы влезть не сможет.

Важные методы Context:

- `getFilesDir()` - путь до приватной директории приложения
- `getDir(String name, int mode)` - открывает/создает директорию в приватном хранилище
- `getCacheDir()` - путь до директории для хранения кэшей
- `deleteFile(String name)` - удаляет приватный файл
- `fileList()` - список приватных файлов

company.vk.lesson07	drwxrwx--x	2024-11-17 15:02	4 KB
> cache	drwxrws--x	2024-11-17 15:25	4 KB
> code_cache	drwxrws--x	2024-11-17 15:02	4 KB
> databases	drwxrwx--x	2024-11-17 15:25	4 KB
> files	drwxrwx--x	2024-11-17 15:53	4 KB
> shared_prefs	drwxrwx--x	2024-11-17 15:25	4 KB

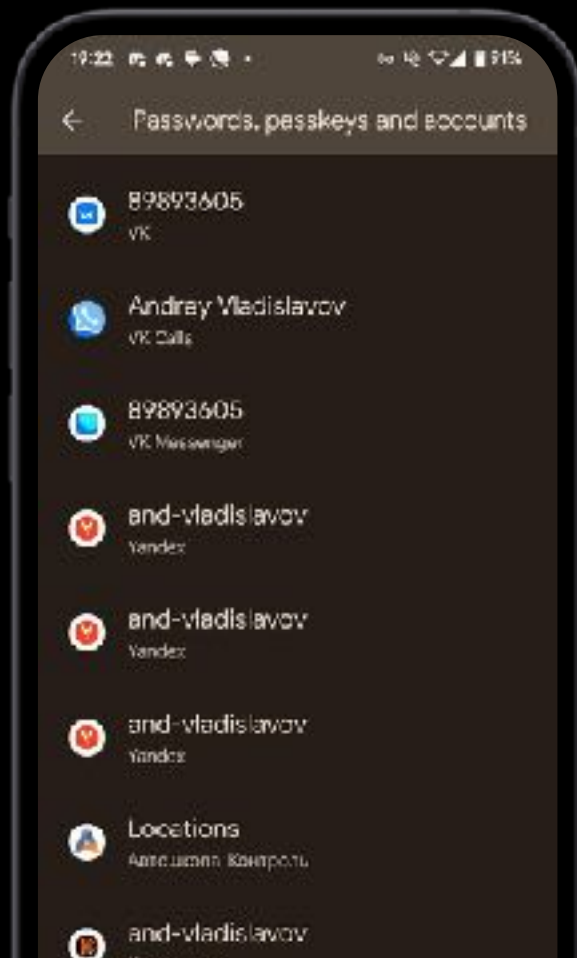
AccountManager

AccountManager

- Переиспользование данных между приложениями
- Тесная интеграция в системе
- Сложно реализовывать

Но

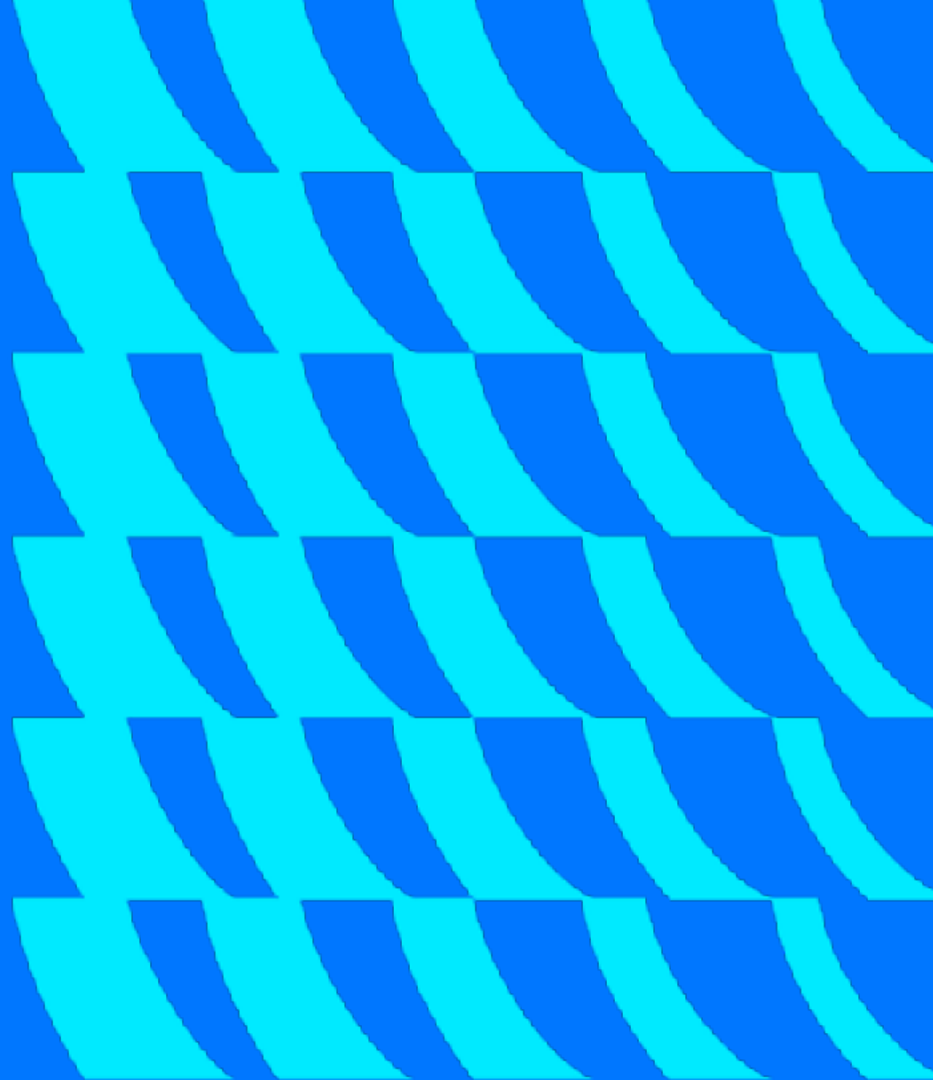
- Плохая документация
- Работает очень нестабильно
- Нельзя работать с ним на главном потоке (хотя Google говорит обратное)
- Не отследить удаление аккаунта юзером



AccountManager. Использование

1. Зарегистрировать пермишены для работы с аккаунтами
2. Реализовать `AbstractAccountAuthenticator`
3. Реализовать `Service` для предоставления аутентификатора
4. Если необходима фоновое обновление данных - то еще реализовать и `SyncAdapter`
5. И фиксировать возникающие проблемы:)

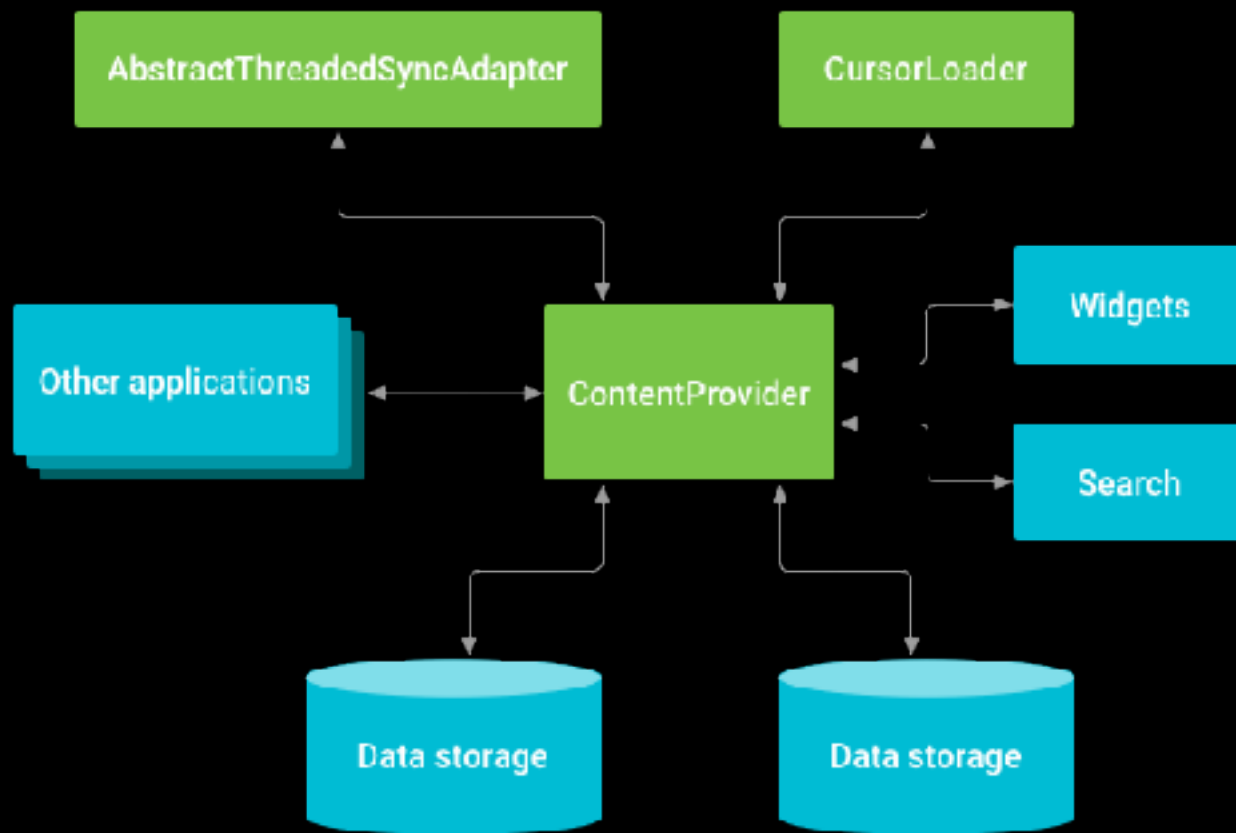
Content
Provider



Один из основных
компонентов Android

Старт приложения





ContentProvider

Интерфейс для доступа к данным приложения.
Как из своего приложения так и из другого.

Хранение данных остается на совести разработчика.

Инициализируется для одного процесса (?), но может данные передавать разным процессам, в виде Cursor.

Создается до Application.onCreate! И эту особенность можно использовать в разных целях.

Важно заполнить поля:

- **name** - сам класс с именем пакета
- **authorities** - идентификатор для поиска провайдера. Должен быть уникальным.

```
<provider android:authorities="list"
  android:directBootAware=["true" | "false"]
  android:enabled=["true" | "false"]
  android:exported=["true" | "false"]
  android:grantUriPermissions=["true" | "false"]
  android:icon="drawable_resource"
  android:initOrder="integer"
  android:label="string_resource"
  android:multiprocess=["true" | "false"]
  android:name="string"
  android:permission="string"
  android:process="string"
  android:readPermission="string"
  android:syncable=["true" | "false"]
  android:writePermission="string" >
  ...
</provider>
```

Имплементация... ох....

Необходимо имплементировать:

- **onCreate()** - вызывается при инициализации
- **query**(Uri, String, Bundle, CancellationSignal) - для возврата данных
- **insert**(Uri, ContentValues) - для записи данных
- **update**(Uri, ContentValues, Bundle) - для обновления данных
- **delete**(Uri, Bundle) - для удаления данных
- **getType**(Uri) - для определения MIME-типа данных

Взаимодействие идет через Context.contentResolver.

Основные методы:)

- **query** - для получения данных
- **insert** - для вставки данных
- **update** - для обновления данных
- **delete** - для удаления данных

Методы имеют вариативные параметры, которые управляют данными, сортировкам.

Еще раз вспомним про особенности

Запускается до старта приложения!

Эта особенность используется
фреймворками для ранней
инициализации:

- Фреймворк для ловли крэшей
- Инициализация некоторых систем аналитик
- Инициализация компонентов

Обратить внимание на фреймворки:

- Firebase Crashlytics
- androidx App Startup

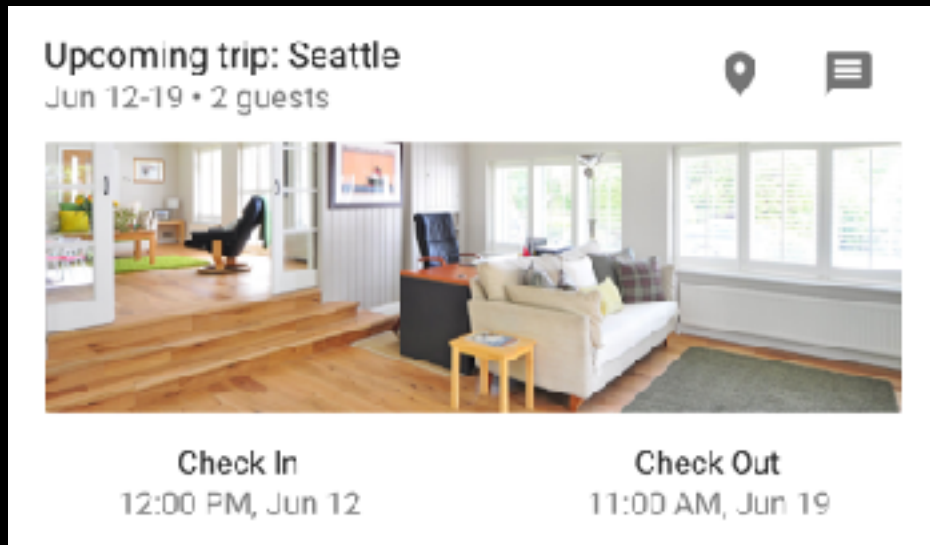


Еще интересное использование - SliceProvider

Предоставление системе контента в виде Слайсов.

Данный компонент скорее стоит воспринимать как пример развития ContentProvider. К сожалению, после анонса, Слайсы не были сильно востребованы, и сейчас в системе увидеть их не так просто.

```
public MySliceProvider extends SliceProvider {  
    public Slice onBindSlice(Uri sliceUri) {  
        String path = sliceUri.getPath();  
        switch (path) {  
            case "/weather":  
                return createWeatherSlice(sliceUri);  
            case "/traffic":  
                return createTrafficSlice(sliceUri);  
        }  
        return null;  
    }  
}
```



Спасибо за
внимание!

